



Universidad de Concepción

Facultad de Ingeniería

Departamento de Ingeniería Eléctrica

Taller de Integración TIC

TIC3

Profesor: Luciano Radriagán

Informe Tarea 1

Sistema de adquisición ESP32-S3 y BMI270
gestionado desde Raspberry Pi

Diego Sanhueza Chávez
Vicente Vásquez Garrido

Concepción, 21 de octubre de 2026

1 Arquitectura servidor e interfaz

El sistema adquiere aceleración y velocidad angular con BMI270 y ESP32-S3; la Raspberry funciona como AP y servidor. La GUI permite configurar, iniciar y detener sesiones. Las capturas verifican esta arquitectura [1]. Las muestras van directamente de la ESP a la Raspberry; el PC accede a la interfaz y administra el equipo.

BMI270 → I2C y FIFO → ESP y arreglos → WiFi → servidor y disco

Servicio en la Raspberry	Función
5001 TCP o UDP	Recibir bloques de muestras en binario o JSON
5000 TCP	Intercambiar estado, órdenes, configuración y previsualización
8080 HTTP	Servir la GUI y su API; acceso AP en http://192.168.50.1:8080

Recepción y sesiones en el servidor

El servidor Python atiende conexiones TCP y datagramas UDP en tareas separadas. Comprueba identidad, cabeceras, tamaños y CRC; reconstruye fragmentos por dispositivo, arranque, sensor y ventana. TCP requiere reconstruir mensajes porque una lectura del socket puede contener una parte o varios mensajes. Los bloques completos se guardan; los incompletos expiran y se contabilizan. Cada inicio crea una sesión con configuración, fechas, cantidades y resultado. En TCP, el ACK TIC3 se emite después del guardado durable.

Qué muestra y qué controla la GUI

La página se sirve localmente, sin recursos externos. Muestra conexión e identidad de la ESP, estado de captura, configuración efectiva, carpeta de salida, errores y sesiones. Las gráficas XYZ expresan aceleración en m/s² y giro en rad/s. El navegador consulta /api/status cada 200 ms; el control entrega preview hasta 10 veces/s. Esa vista resume la adquisición: sus contadores no certifican persistencia. Los totales de almacenamiento son acumulados del servidor; la tabla de sesiones permite revisar cada ejecución.

Cómo se aplican los parámetros

Con la captura detenida se eligen tasas y rangos de ambos sensores, ventana, objetivos XYZ, modo secuencial/paralelo, TCP/UDP y binario/JSON. Aplicar configuración envía un POST a /api/config; el servidor valida y asigna una revisión. La ESP recibe CONFIG por 5000, reserva memoria, aplica cambios y verifica registros. Solo CONFIG_OK con revisión y valores coincidentes actualiza la configuración efectiva. Elegir un desplegable no modifica el sensor. Un fallo se informa y se intenta restaurar la configuración anterior; cambiar WiFi requiere configurar el firmware.

Iniciar captura envía START. Detener y guardar envía STOP: la ESP cierra el bloque parcial y procesa pendientes antes de confirmar el cierre TCP. Los objetivos son muestras XYZ adquiridas; cero indica captura continua. En UDP la GUI conserva un resultado sin confirmación de entrega, aunque el servidor haya recibido bloques.

Muestreo ventana y visualización

A 400 Hz hay una XYZ nominal cada 2,5 ms por sensor. La ESP vacía FIFO aproximadamente cada 10 ms; una ventana de 1000 ms produce un arreglo por sensor/s con unas 400 XYZ. El envío depende de colas y ACK, y puede fragmentarse. El control pausa nominalmente 100 ms. read_us marca lectura FIFO, puede repetirse entre muestras y no es UTC ni el instante exacto de conversión.

2 Decisiones de diseño

WiFi por prioridades y continuidad

La ESP evalúa tres perfiles en orden: AP de la Raspberry, laboratorio y red común. Cada perfil contiene credenciales y dirección del servidor. Escanea redes visibles y prueba el primer perfil habilitado y válido, con dos intentos de asociación. Si falla, lo excluye 60 s y prueba el siguiente; sin candidatos, repite con espera creciente hasta 30 s y una variación aleatoria. Tres fallos consecutivos de acceso al servidor también provocan cambio: tener IP no demuestra que el receptor esté disponible.

Se conserva una conexión saludable aunque aparezca un AP preferido, evitando cortes durante la captura. Los cambios de enlace/IP invalidan sockets anteriores mediante una generación de conexión. El AP permite operar sin router externo; las otras redes ofrecen alternativas. Los perfiles WiFi se configuran en firmware.

Paralelismo memoria y control seguro

En paralelo, adquisición ocupa core 1 y salida core 0; las colas transfieren bloques inmutables para que la espera de red no bloquee normalmente la captura. En secuencial, adquisición transmite al cerrar la ventana y deshabilita la entrada a FIFO durante ese envío; después retoma.

Hay dos buffers por flujo y un presupuesto inicial de 128 KiB para arreglos; a 400/400 Hz y ventana de 1 s se reservan 23968 bytes. Sin bloque libre, se conserva lo pendiente y se descarta la ventana nueva, registrando el descarte. Aplicar parámetros solo en reposo y reservar memoria antes de sustituir buffers evita mezclar escalas o tasas en una misma ventana.

Transporte confirmaciones y recuperación

TCP es el valor inicial por orden y retransmisión. El ACK TIC3 confirma guardado: la ESP conserva el bloque hasta recibirlo y puede reenviarlo tras reconectar. La identidad del bloque y una huella lógica permiten deduplicar incluso tras reiniciar el servidor. UDP conserva datagramas, pero no incorpora ACK ni recuperación por pérdida silenciosa. El control permanece en TCP 5000. Un ACK TCP confirma bytes; no acredita escritura en disco ni recupera muestras descartadas antes del envío.

Binario y almacenamiento durable

El binario usa orden de bytes y tamaños explícitos: 96 bytes de cabecera y 14 por XYZ, con secuencia, offset temporal y tres int16. Admite 78 XYZ por fragmento, frente a seis en JSON: 400 XYZ requieren seis fragmentos binarios o 67 JSON. TCP antepone cuatro bytes de longitud; sus segmentos no delimitan mensajes. El receptor guarda binario en .bin mediante TIC3BIN y JSON en .json; ambos conservan metadatos y pueden leerse con load_block.

Estado y manifiestos siguen en JSON por facilidad de inspección, sin convertir las muestras binarias a texto. Se escribe un temporal, se ejecuta fsync, se reemplaza atómicamente y se sincroniza el directorio antes del ACK. Ese costo permite confirmar persistencia. Separar datos, control y HTTP permite atender la GUI mientras se reciben bloques.

Validación y límites de seguridad

Se validan parámetros, memoria, longitudes, fragmentos y CRC32; los .bin agregan SHA256. La API comprueba origen y cabecera de control, y limita solicitudes; las credenciales quedan fuera del repositorio. WPA2/AES protege el AP. CRC y SHA256 detectan corrupción, pero no autentican al emisor. HTTP carece de TLS y autenticación fuerte: el uso previsto es una red local controlada.

3 Efecto de frecuencia y rango del BMI270

Se analizaron las cinco sesiones históricas con manifiestos y 44 bloques del directorio sensores/ [3]. Conservan cuentas raw, escala, frecuencia y rango por bloque. La tabla compara cuatro combinaciones distintas; las cantidades corresponden a muestras reconstruidas y guardadas en esos archivos, no a puntos de la gráfica de preview.

Sesión	Hz acc / giro	Rangos ± g / °/s	XYZ acc / giro	Media Z raw	Media |a| m/s²
ec15ed5a	200 / 100	2 / 250	300 / 150	16477,2	9,913
6f481649	100 / 50	4 / 500	250 / 125	8234,7	9,911
881331db	100 / 100	8 / 2000	50 / 50	4115,8	9,902
70833adb	50 / 25	16 / 1000	120 / 60	2090,3	10,058

Fuente: cálculo sobre window-*.json de las sesiones indicadas. La quinta sesión f8b45ad9 usa 100/50 Hz, ±4 g/±500 °/s y guarda 1000/500 XYZ; su media |a| es 9,911 m/s². Las sesiones tienen distinta duración y algunas ventanas finales son parciales.

Frecuencia de muestreo

A 50, 100 y 200 Hz, los períodos nominales son 20, 10 y 5 ms. Con una ventana de 500 ms se esperan unas 50 XYZ a 100 Hz o 100 XYZ a 200 Hz. En las sesiones 6f481649 y ec15ed5a se observaron 49 a 51 y 99 a 101 XYZ por bloque de aceleración, respectivamente. El conteo real varía con las ventanas y el cierre por objetivo. Una frecuencia mayor mejora la resolución temporal disponible, a cambio de más registros, RAM y tráfico.

El máximo de frecuencia representable sin ambigüedad exige considerar Nyquist y el filtrado; fs/2 es un límite teórico, no una garantía de banda útil. Aumentar la tasa no corrige sesgo ni asegura mejor exactitud. Los timestamps de lectura FIFO permiten estudiar conteos y ventanas, pero no verificar jitter físico de cada conversión.

Rango y sensibilidad digital

El acelerómetro de 16 bits permite ±2, ±4, ±8 y ±16 g [2]. Sus sensibilidades nominales son 16384, 8192, 4096 y 2048 cuentas/g. El paso digital correspondiente es 0,5986; 1,1971; 2,3942 y 4,7884 mm/s² por cuenta. Pasar de ±2 a ±16 g multiplica por ocho la amplitud medible y también el tamaño del paso digital.

La media Z raw cambia aproximadamente de 16477 cuentas a ±2 g a 8235 a ±4 g, 4116 a ±8 g y 2090 a ±16 g. Al aplicar la escala, las magnitudes medias permanecen alrededor de 9,9 a 10,1 m/s²: el cambio de cuentas refleja la escala, no una aceleración física ocho veces menor. Para giro se conserva la misma lógica: ampliar el rango aumenta los rad/s representados por cuenta.

Un rango pequeño distingue incrementos digitales más finos, pero un eje que exceda el fondo de escala se satura y pierde la amplitud real del pico. Un rango grande reduce ese riesgo a costa de una cuantización más gruesa. El firmware no incorpora un detector específico de saturación.

Alcance experimental

Estas sesiones cambian simultáneamente rango, tasa, duración, modo y transporte; no reproducen un movimiento de referencia controlado. Por ello permiten verificar escalas y conteos, pero no atribuir diferencias de media o ruido exclusivamente al rango o a la frecuencia. No se presenta esta comparación como calibración ni ensayo controlado de exactitud.

4 Comparación TCP y UDP con Wireshark

T01 y T02 se capturaron en wlan0 del AP de la Raspberry el 21 de octubre: 400/400 Hz, ±8 g/±2000 °/s, ventana de 1 s, paralelo, binario y objetivo 12000 XYZ/sensor. Los mensajes se reensamblaron y validaron con el codec TIC3 [4].

Medición en el canal de datos	T01 TCP	T02 UDP
Filtro aplicado	tcp.port == 5001	udp.port == 5001
Paquetes / datagramas observados	734	356
Bytes de tramas capturadas	417744	385128
Mensajes DATA TIC3	356	356
Bloques completos acc / giro	30 / 30	30 / 30
XYZ reconstruidas acc / giro	12000 / 12000	12000 / 12000
ACK TIC3 de guardado observados	60	No implementados

Tramas del PCAP en ambas direcciones, no bytes de radio. TCP incluye conexión y ACK; UDP excluye dos TCP residuales. No se cuentan control 5000, HTTP ni SSH.

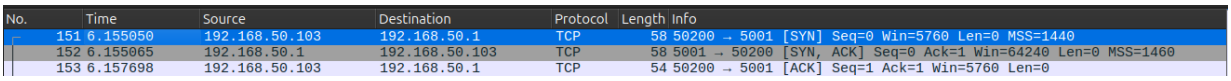


Figura 1. T01: SYN, SYN/ACK y ACK entre ESP 192.168.50.103 y Raspberry 192.168.50.1, puerto de datos 5001.

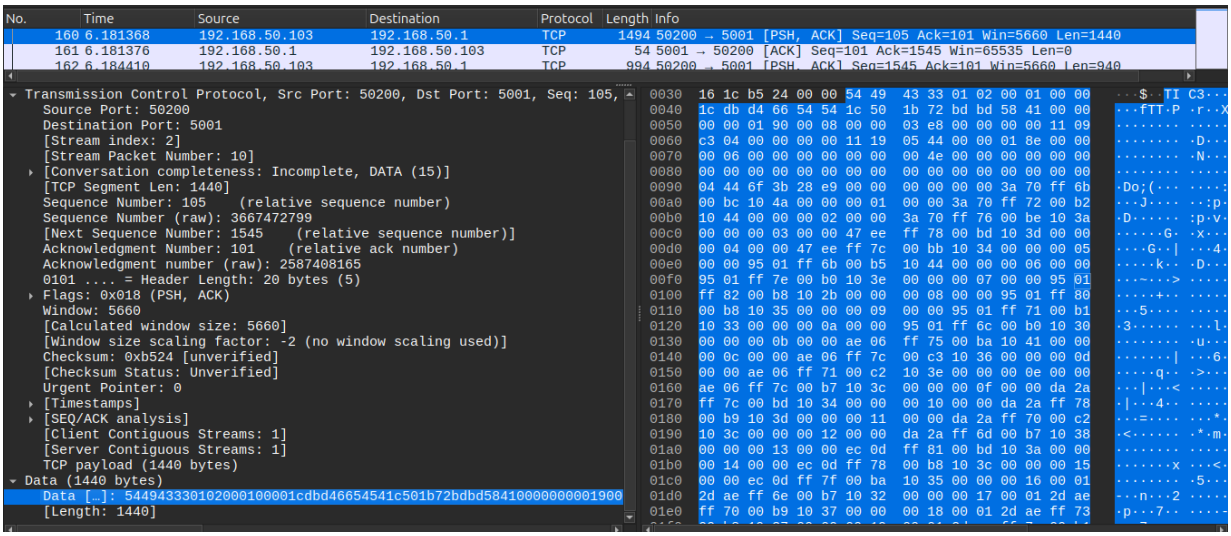


Figura 2. T01: paquete 160 ESP → Raspberry:5001, [PSH, ACK], Len=1440 y Data desplegado en hexadecimal. Es payload binario; el ACK 161 tiene Len=0. Un segmento puede contener parte de un mensaje TIC3.

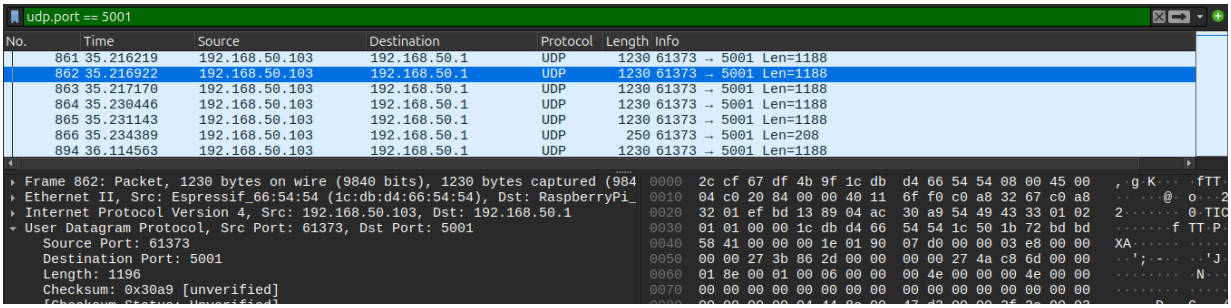


Figura 3. T02: UDP a 5001; payload TIC3 de 1188 bytes y longitud UDP de 1196 bytes con cabecera.

El control permanece en TCP 5000. El ACK TCP reconoce bytes, no guardado; UDP carece de ACK de transporte. UDP registró 7,8 % menos bytes de tramas; la franja DATA fue 28,80/28,83 s, sin medir latencia muestra a disco.

5 Interpretación y resultados de red

Cómo se distinguen TCP y UDP en las capturas

En T01, los paquetes 151 SYN, 152 SYN/ACK y 153 ACK establecen la conexión. El 160 lleva [PSH, ACK], Seq=105 y Len=1440: transporta 1440 bytes; el 161 responde Ack=1545 y Len=0, indicando el siguiente byte esperado (105 + 1440). PSH solicita que se entregue lo recibido a la aplicación; no delimita un arreglo ni garantiza guardado. La columna Length incluye cabeceras, mientras Len indica payload TCP. Data y el panel hexadecimal muestran bytes del protocolo TIC3; sin su decodificador no aparecen las variables XYZ por nombre.

En T02 se observan datagramas UDP hacia 5001, sin handshake, Seq/Ack ni confirmaciones de ese transporte. El paquete 862 contiene 1188 bytes TIC3; UDP Length=1196 incluye ocho de cabecera y la trama mide 1230. Un datagrama contiene un fragmento, no necesariamente un arreglo entero. El control sigue por TCP 5000: verlo en una prueba UDP es correcto. Allí RSL [Malformed Packet] corresponde a una interpretación ajena al protocolo TIC3; no prueba corrupción de muestras. Deshabilitar el disector IPA/RSL permite inspeccionar los bytes como datos [5].

Resultados adicionales y alcance de las imágenes

T03 (TCP/binario/secuencial) entregó 12000/12000 XYZ; su franja DATA fue 34,50 s frente a 28,80 s de T01, compatible con pausas de adquisición. T04 (UDP/binario/secuencial) permitió reconstruir 12000/11601 XYZ: faltan 399 de giro (3,325 %) y queda incompleta la ventana 110. La captura no permite localizar la pérdida en envío, enlace o captura.

T05 (TCP/JSON/paralelo) entregó 12000/12000 XYZ: 4474 paquetes y 2942385 bytes, 7,04 veces los bytes de T01. Esto mide representación y fragmentación, no una aceleración de siete veces. Su pantallazo Follow TCP Stream muestra únicamente el sentido servidor → ESP, con ACK TIC3 binarios; para comprobar JSON debe elegirse ESP → servidor. Los puntos de la vista ASCII representan bytes no imprimibles, no pérdida ni cifrado [5].

Conclusiones y límites

Paralelo y binario favorecen continuidad y representación compacta; TCP permite confirmar persistencia mediante ACK TIC3, diferente del ACK TCP. UDP reduce confirmaciones y no recupera bloques incompletos. Tasa y rango deben adaptarse al movimiento para obtener resolución temporal suficiente y evitar saturación.

Las pruebas no son ensayos estadísticos repetidos. Los totales de GUI son acumulados y las sesiones históricas de sensibilidad pertenecen a otra campaña. No se midieron energía, vida útil de SD ni latencia absoluta. Los 15 pantallazos y sus breves descripciones se conservan en la carpeta wireshark/.

Referencias

[1] Tarea 1 - Taller de Integración TIC (TIC3). Enunciado del curso, 22-09-2026. RF1-RF9 y entregables.

[2] Bosch Sensortec. BMI270 Datasheet BST-BMI270-DS000-08, revisión 1.6, marzo de 2026. Secciones de especificaciones y procesamiento de datos. https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmi270-ds000.pdf

[3] Evidencia del proyecto: sensores/sessions/*.json y 44 archivos window-*.json; configuración, escalas y conteos verificados por sesión. Fuentes: esp/main y raspi/.

[4] Capturas del AP: wireshark/ap/T01 a T05, 21-10-2026. Cifras obtenidas al filtrar el transporte y puerto 5001, reensamblar los mensajes TIC3 y verificar conteos y CRC con el receptor del proyecto.

[5] Wireshark User's Guide. TCP Analysis, Following Protocol Streams y Control Protocol Dissection. https://www.wireshark.org/docs/wsug_html_chunked/ (capítulos 7 y 11).